

Team 7 - E-Waste Battery Extraction

CV-21 Part A: Instrumentation & Baseline Benchmarking

Instrumentation and Baseline Results

ROS 2 Jazzy, Gazebo, ONNX Runtime and Performance Instrumentation

Ubuntu 24.04 on WSL2

Instrumentation and baseline report

September 2026

Contents

1	Purpose of CV-21	2
2	Relationship to Other CV Tasks	2
3	Project and Environment Paths	2
4	Terminal Convention Used	3
5	Step 1: Preserve the Original Baseline	3
6	Step 2A: Instrumentation of <code>inference.py</code>	4
7	Step 2B: Instrumentation of the ROS Detector Node	5
8	Indentation and Syntax Issue Encountered	5
9	Step 3: Rebuild and Launch	6
10	Step 4: Baseline Data Collection	7
11	Steady-State Benchmark Method	7
12	Baseline Statistical Results	8
13	Main Technical Findings	8
14	CV-17 Self-Check Outlier	9
15	FPS Interpretation	9
16	Evidence Produced in Part A	10
17	Meaning of the Main Evidence Files	10
18	State at the End of Part A	11
19	Preparation for Part B	11

1 Purpose of CV-21

CV-21 is the **Inference Performance Profiling & Optimisation** task for the computer vision pipeline. The task is concerned with measuring where time is spent in the perception pipeline, identifying bottlenecks, and later evaluating practical optimisation strategies.

The main stages relevant to CV-21 are:

- image conversion and preprocessing;
- ONNX Runtime inference;
- post-processing;
- annotation/debug processing;
- image saving;
- ROS message construction and publishing;
- total callback latency and effective processing FPS.

This chat covered **Part A only**: instrumentation and baseline benchmarking. No optimisation was intentionally applied during the benchmark.

2 Relationship to Other CV Tasks

CV-21 can be performed without first completing CV-18, CV-19, or CV-20.

The main practical prerequisite is that the existing perception pipeline is already operational. In this project, the required pieces were already available from the earlier ROS 2, Gazebo, preprocessing, ONNX inference, and localisation work.

CV-21 complements CV-17 rather than replacing it:

- **CV-17** focuses on correctness of pixel-to-world localisation and validation against Gazebo ground truth.
- **CV-21** focuses on runtime behaviour, latency, throughput, bottleneck identification, and later optimisation.

Therefore, CV-17 contributes mainly to geometric correctness and robotics perception, while CV-21 contributes to performance engineering and deployability.

3 Project and Environment Paths

The project is stored on the Windows E: drive and accessed from WSL through:

`/mnt/e/team7_project_bundle/`

The main perception source directory used in Part A was:

```
/mnt/e/team7_project_bundle/team7_perception_ws/src/  
team7_perception/team7_perception/
```

The important source files were:

```
geometry.py  
inference.py  
onnx_phone_detector_node.py
```

The CV-21 benchmark evidence directory was created at:

```
/mnt/e/team7_project_bundle/cv21_benchmark/baseline/
```

4 Terminal Convention Used

For consistency, the following terminal convention was used:

- **Terminal 1:** Gazebo / simulation.
- **Terminal 2:** ONNX perception detector.
- **Terminal 3:** benchmark checking, CSV inspection, statistics, and analysis.

This convention should be retained in Part B unless there is a clear reason to change it.

5 Step 1: Preserve the Original Baseline

Before modifying the code, untouched baseline copies were created:

```
inference_cv21_baseline.py  
onnx_phone_detector_node_cv21_baseline.py
```

These were stored alongside the original source files in the perception package.

A baseline system information file was also created:

```
/mnt/e/team7_project_bundle/cv21_benchmark/baseline/  
system_info.txt
```

The recorded environment was:

- Python: 3.12.3
- ONNX Runtime: 1.29.0
- Available ONNX Runtime providers:
 - AzureExecutionProvider
 - CPUExecutionProvider
- CPU: Intel Core i5-9300H @ 2.40 GHz
- Physical CPU cores: 4
- Logical CPUs: 8
- WSL memory available: approximately 15 GiB

CUDAExecutionProvider was not available in the baseline environment. This was intentionally left unchanged so that Part A represented the real existing system.

6 Step 2A: Instrumentation of `inference.py`

The `inference.py` file was instrumented using `time.perf_counter()`.

The `infer()` function was extended so that it can optionally return timing data while keeping compatibility with the existing caller.

The added timing measurements were:

```
preprocess_ms  
inference_ms  
postprocess_ms
```

The function retained its normal behaviour when timing output is not requested, using a parameter equivalent to:

```
return_timings=False
```

When timing output is requested, the function returns both detections and a timing dictionary.

The main purpose was to isolate:

1. preprocessing latency;
2. ONNX `session.run()` latency;
3. post-processing latency.

7 Step 2B: Instrumentation of the ROS Detector Node

The file:

```
onnx_phone_detector_node.py
```

was instrumented to measure the complete callback and the major stages surrounding model inference.

A configurable performance CSV path was added so that benchmark output could be written directly into the CV-21 evidence directory.

The main measurements added were:

```
bridge_ms  
raw_save_ms  
preprocess_ms  
inference_ms  
postprocess_ms  
annotation_ms  
annotated_save_ms  
message_build_ms  
metrics_csv_ms  
publish_ms  
self_check_ms  
callback_total_ms  
processing_fps
```

The extra `self_check_ms` metric was included because the existing CV-17 geometry self-check is executed from the image callback and can invoke a Gazebo ground-truth query. Without measuring it separately, the first-frame callback could appear to be an unexplained performance outlier.

The new profiling CSV write itself was intentionally performed after `callback_total_ms` was measured so that benchmark logging did not contaminate the measured callback latency.

8 Indentation and Syntax Issue Encountered

During editing in Nano, a mixture of tabs and spaces caused:

```
TabError: inconsistent use of tabs and spaces in indentation
```

After converting leading tabs to spaces, a second error appeared:

```
IndentationError: unexpected indent
```

The problem was traced to the new performance CSV initialisation block, where several lines had only four leading spaces instead of eight inside `__init__()`.

The block was rewritten with the correct indentation. After the fix:

```
python3 -m py_compile inference.py
python3 -m py_compile onnx_phone_detector_node.py
```

both completed with no output, confirming valid Python syntax.

9 Step 3: Rebuild and Launch

The ROS workspace was rebuilt using:

```
cd /mnt/e/team7_project_bundle/team7_perception_ws
colcon build --symlink-install
```

The build completed successfully:

```
Summary: 2 packages finished
```

There were clock-skew warnings from files on the Windows-mounted E: drive, but these were warnings rather than build failures.

Gazebo was started from Terminal 1 using the supplied project launcher.

The instrumented detector was launched from Terminal 2 with the performance CSV path explicitly set to:

```
/mnt/e/team7_project_bundle/cv21_benchmark/baseline/
performance_metrics.csv
```

At runtime, ONNX Runtime reported:

```
CPUExecutionProvider
```

A warning also confirmed that `CUDAExecutionProvider` was not available. This matched the environment recorded in `system_info.txt`.

10 Step 4: Baseline Data Collection

The instrumented detector successfully produced a CSV containing one row per processed frame.

The CSV header contained:

```
stamp
detection_count
bridge_ms
raw_save_ms
preprocess_ms
inference_ms
postprocess_ms
annotation_ms
annotated_save_ms
message_build_ms
metrics_csv_ms
publish_ms
self_check_ms
callback_total_ms
processing_fps
```

A total of 221 measured frames were collected, plus the CSV header.

The raw collected file was preserved as:

```
performance_metrics_raw.csv
```

This frozen copy should remain untouched and should be used as the authoritative Part A baseline dataset.

11 Steady-State Benchmark Method

The first 20 frames were excluded from the steady-state statistical analysis.

They were **not deleted**. They remain present in the frozen raw CSV as evidence of startup/cold behaviour.

The steady-state dataset therefore contained:

```
221 total measured frames
- 20 startup frames
= 201 steady-state frames
```

This allowed the benchmark to distinguish normal runtime behaviour from one-off startup effects.

12 Baseline Statistical Results

The steady-state benchmark produced the following overall results:

Metric	Result
Steady-state frames	201
Mean callback latency	545.371 ms
Median callback latency	518.476 ms
P95 callback latency	825.621 ms
Mean processing FPS	1.943 FPS
Median processing FPS	1.929 FPS
Maximum observed processing FPS	3.143 FPS

The main per-stage steady-state results were:

Stage	Mean latency	Share of callback
ONNX inference	374.840 ms	68.73%
Annotation drawing	67.998 ms	12.47%
Raw PNG saving	34.424 ms	6.31%
Annotated PNG saving	31.026 ms	5.69%
Post-processing	19.352 ms	3.55%
Preprocessing	12.474 ms	2.29%
ROS image conversion	4.137 ms	0.76%
Message construction	0.361 ms	0.07%
Metrics CSV writing	0.295 ms	0.05%
ROS publishing	0.286 ms	0.05%
CV-17 self-check	0.002 ms	approximately 0.00%

13 Main Technical Findings

The strongest conclusion from Part A is that **ONNX inference is the dominant runtime bottleneck**, consuming approximately 68.73% of the steady-state callback time.

The next largest cost is annotation drawing at approximately 12.47%.

The two PNG-saving stages together contribute approximately:

$$6.31\% + 5.69\% = 12.00\%.$$

Therefore, annotation and per-frame image saving together account for roughly 24% of total callback time.

By contrast, ROS message construction and ROS publishing are negligible contributors to the total latency.

This gives Part B a clear optimisation direction:

1. investigate ONNX Runtime inference configuration;

2. investigate unnecessary debug/annotation work;
3. investigate per-frame image saving;
4. compare thread settings;
5. test warm-up effects;
6. evaluate the best combined configuration.

14 CV-17 Self-Check Outlier

The first portion of the run contained a large one-off callback outlier.

Across all recorded frames:

maximum callback latency = 2571.722 ms

The corresponding CV-17 geometry self-check took approximately:

2079.235 ms

In steady state, however:

mean self_check_ms = 0.002 ms

This demonstrates that the large early callback was caused by a one-time CV-17 validation operation rather than by normal inference.

The self-check warning also reported a mismatch between projected and Gazebo ground-truth position. This did not block CV-21 because Part A was focused on performance measurement rather than localisation correctness.

15 FPS Interpretation

The reported mean FPS and reciprocal of mean latency are not expected to be identical.

The benchmark stores per-frame FPS as:

$$\text{FPS}_i = \frac{1000}{\text{latency}_i}.$$

The reported mean processing FPS is therefore:

$$\frac{1}{N} \sum_{i=1}^N \frac{1000}{\text{latency}_i},$$

which is different from:

$$\frac{1000}{\text{mean latency}}.$$

For Part B and the final report, callback latency should be treated as the primary performance metric, with FPS used as a complementary throughput measure.

16 Evidence Produced in Part A

The final baseline evidence directory contains:

```
/mnt/e/team7_project_bundle/cv21_benchmark/baseline/  
  
system_info.txt  
performance_metrics.csv  
performance_metrics_raw.csv  
benchmark_summary.txt  
baseline_stage_latency.png  
baseline_callback_latency.png  
inference_instrumented.py  
onnx_phone_detector_node_instrumented.py
```

The instrumented source files were copied into the benchmark directory after the experiment so that the exact implementation used to generate the baseline data is preserved.

17 Meaning of the Main Evidence Files

- `system_info.txt`: records the machine and ONNX Runtime environment.
- `performance_metrics.csv`: working benchmark CSV.
- `performance_metrics_raw.csv`: frozen copy of the exact raw baseline dataset.
- `benchmark_summary.txt`: statistical summary of all frames and the 201-frame steady-state subset.
- `baseline_stage_latency.png`: visual comparison of mean latency by processing stage.
- `baseline_callback_latency.png`: callback latency across steady-state frames with mean and P95 reference lines.
- `inference_instrumented.py`: exact instrumented inference source used for Part A.
- `onnx_phone_detector_node_instrumented.py`: exact instrumented ROS node used for Part A.

18 State at the End of Part A

At the end of this chat:

- baseline source files had been preserved;
- instrumentation had been added successfully;
- both Python files compiled without syntax errors;
- the ROS workspace built successfully;
- Gazebo and the detector ran successfully;
- the runtime provider was confirmed as CPUExecutionProvider;
- 221 baseline frames were collected;
- 201 steady-state frames were analysed;
- the raw dataset was frozen;
- summary statistics were generated;
- two evidence plots were generated;
- the exact instrumented source files were archived.

Therefore, **CV-21 Part A is complete.**

19 Preparation for Part B

Part B should start from the frozen baseline and change **one factor at a time** before testing a final combined configuration.

A suitable sequence is:

1. warm-up experiment;
2. disable or reduce per-frame raw/annotated PNG saving;
3. make annotation/debug output optional;
4. test ONNX Runtime CPU thread settings, for example 1, 2, and 4 intra-op threads;
5. explicitly compare provider/runtime settings available in the environment;
6. investigate avoidable image copies or repeated work;
7. identify the best individual changes;
8. create a final combined optimised configuration;
9. rerun the same benchmark protocol;
10. compare final latency and FPS directly against the Part A baseline.

The frozen Part A reference values to compare against are:

Mean callback latency:	545.371 ms
Median callback latency:	518.476 ms
P95 callback latency:	825.621 ms
Mean processing FPS:	1.943 FPS
ONNX inference share:	68.73%

These values should remain unchanged and should be used as the formal baseline when evaluating Part B optimisation results.